

Resolução Prob com Grafos - T290 - Prof. Carubbi

NUMBER MAZE

Uma abordagem usando o algoritmo de Dijkstra

GABRIEL DE SOUSA NOBRE
VICTOR LINS GURGEL DO AMARAL
LORENZO BARROS CALHEIROS PINHEIRO

- ✦ **Modelagem do Problema**
- ✦ **Solução Proposta**
- ✦ **Implementação do Algoritmo**



Universidade
de Fortaleza

Explicação do Problema

Encontrar em um labirinto (em formato de uma matriz) o menor caminho possível, começando da **POSIÇÃO SUPERIOR ESQUERDA** e chegando até a **POSIÇÃO INFERIOR DIREITA** usando o algoritmo de Dijkstra:

O Grafo:

- Vértices seriam as casas da matriz (posições da matriz)
- Há existência de Arestas em casas adjacentes (até 4 direções), e:
 - arestas são direcionadas
 - há sempre uma aresta de entrada e uma de saída em cada vértice adjacente
 - o número presente em cada casa é o peso da aresta incidente ao vértice correspondente

0	3	1	2	9
7	3	4	9	9
1	7	5	5	3
2	3	4	2	5



Universidade
de Fortaleza

Modelagem do Problema

Porque Dijkstra é ideal: No exemplo **todos os pesos das arestas serão sempre positivos**, garantindo a possibilidade de relaxar todas as arestas que a partir de um vértice S e seguir atualizando os custos dos caminhos para chegar em cada vértice

Input:

- X casos teste
- N Números de linhas
- M Números de colunas
- Para cada linha (N):
 - números presentes na linha $n_1, n_2, n_3 \dots$

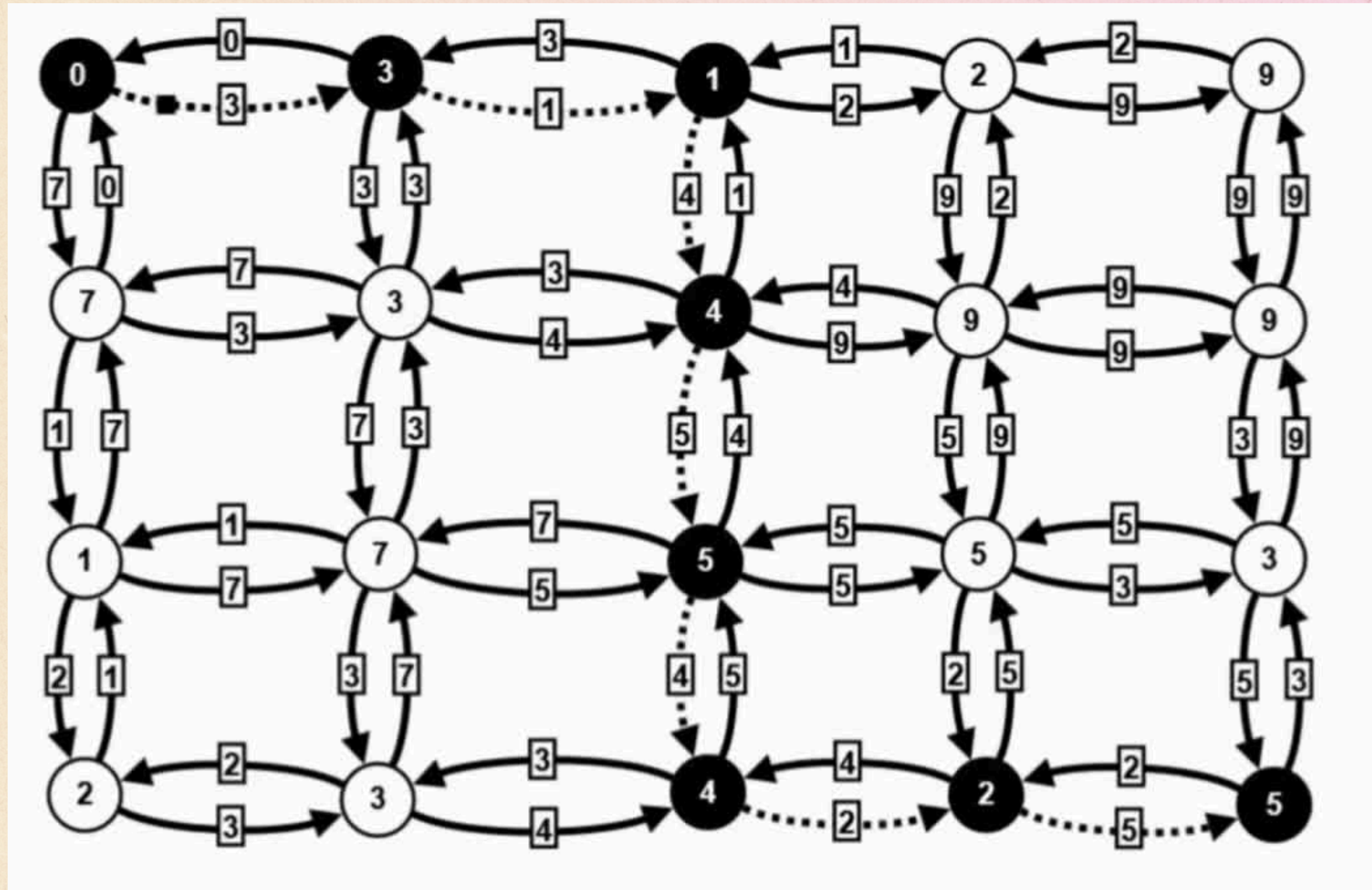
Output:

- Custo mínimo para o caminho

0	3	1	2	9
7	3	4	9	9
1	7	5	5	3
2	3	4	2	5

Solution value: 24

Solução Proposta



GRAPH ONLINE. Grafo exemplo. [S.l.], 2026. Disponível em:
<https://graphonline.top/pt/?graph=qjpMcnrvEBEVsbEKZZcst>.
Acesso em: 27 maio 2026.



Universidade
de Fortaleza

Solução Proposta

LAZY DIJKSTRA: usando priority-queue/heap-mip/buckets;

RELAXAMENTO: novo menor custo encontrado;

SOLUÇÃO FINAL: vertice destino saiu da priority-queue;

CASOS ESPECIAIS: 1×1 ; $w=1 \rightarrow$ BFS; múltiplos caminhos;



Universidade
de Fortaleza

Implementação do Algoritmo

```
1 import sys
2 def solve(input_path):
3     with open(input_path, 'r', encoding='utf-8') as f:
4         input_data = f.read().split()
5
6     data = list(map(int, input_data))
7     num_test_cases = data[0]
8     idx = 1
9     out = []
10
11     for _ in range(num_test_cases):
12         rows, cols = data[idx], data[idx+1]
13         idx += 2
14         size = rows * cols
15         grid = data[idx : idx + size]
16         idx += size
17         _dist = [2147483647] * size
18         _dist[0] = grid[0]
19         buckets = [[] for _ in range(10)]
20         buckets[grid[0] % 10].append(0)
21         d = grid[0]
22         target = size - 1
23         _grid = grid
24
```

```
25     while True:
26         b = buckets[d % 10]
27         while not b:
28             d += 1; b = buckets[d % 10]
29         u = b.pop()
30
31         if _dist[u] < d: continue
32
33         if u == target:
34             out.append(str(d)); break
35
36         if u >= cols: # Cima
37             v = u - cols
38             nd = d + _grid[v]
39             if nd < _dist[v]:
40                 _dist[v] = nd
41                 buckets[nd % 10].append(v)
42
43 >         if u < size - cols: # Baixo...
44
45 >         if u % cols != 0: # Esquerda...
46
47 >         if (u + 1) % cols != 0: # Direita...
48
49 >         sys.stdout.write('\n'.join(out) + '\n')
```

num casos: T

peso max: $W=9*$

inserts na heapq:
 $O(\log V)$

inserts no bucket:
 $O(1)$

$O()$ preliminar:
 $O(T \times E \log V)**$ /
 $O(T \times E + V \times W)$

$O()$ final:
 $O(T \times V \log V)**$ /
 $O(T \times V)$

Espaço: $O(V)$



Universidade
de Fortaleza

* Nosso caso tem custo maximo 9

** Big O se fosse implementação com heapq